

MAVEN EXECUTION MASTER FLOW

A practical lab guide for understanding **what to do, why you do it, and which command to run.**

■ THE MASTER FLOW

1	GitHub repository
2	Clone project
3	Import/open in Eclipse
4	Check pom.xml
5	Fix dependency / compiler / configuration issues
6	Run Maven lifecycle commands
7	Generate JAR or WAR in target/
8	Run or deploy the application
9	Test in browser

QUICK RULE

Exam mindset: Do not memorize random commands. Follow the flow. At every stage, check the output before moving to the next stage.

1. MAVEN — STARTING THE PROJECT

Maven is used to manage the project build, dependencies, compilation, testing and packaging. Your first goal is to get the project into a clean, buildable state.

Step 1 — Clone the repository

Open Command Prompt/Terminal and move to the location where you want the project. Replace `<URL>` with the repository URL given in your question paper.

```
git clone <URL>
cd <project>
```

Step 2 — Import into Eclipse

1	Open Eclipse.
2	Choose File → Import.
3	Select Maven → Existing Maven Projects.
4	Select the cloned project folder.
5	Make sure pom.xml is detected.
6	Click Finish and wait for Maven dependencies to resolve.

Step 3 — Understand pom.xml

pom.xml is the main Maven project configuration file. It tells Maven about the project, its dependencies, Java compiler settings, plugins and packaging/build configuration.

Item	What it means
dependencies	Libraries required by the project.
maven-compiler-plugin	Compiles the Java source code.
maven-war-plugin	Packages a web application as a WAR.
properties	Can define Java source/target and encoding settings.
target/	Contains generated build output such as JAR/WAR.

ECLIPSE CHECK

If your Eclipse project shows **JRE System Library**, **Maven Dependencies**, **src**, **target** and **pom.xml**, Maven has generally recognized the project correctly.

2. MAVEN — EXECUTION COMMANDS

Run commands from the directory that contains **pom.xml**. Use the sequence below when the question asks you to build and verify the project.

Command	Purpose / What to check
<code>mvn -v</code>	Checks Maven installation and Maven version.
<code>java -version</code>	Checks the Java version being used.
<code>mvn clean</code>	Removes previous build output, mainly <code>target/</code> .
<code>mvn compile</code>	Compiles the project source code.
<code>mvn test</code>	Runs tests when required.
<code>mvn package</code>	Builds and packages the project into its artifact.
<code>mvn clean package</code>	Clean old output, then build and package from scratch.
<code>mvn package -DskipTests</code>	Packages while skipping test execution.
<code>mvn clean package -X</code>	Runs a detailed/debug Maven build when troubleshooting.
<code>mvn dependency:tree</code>	Shows the project's dependency hierarchy.
<code>mvn dependency:analyze</code>	Analyzes dependency usage.

Recommended execution sequence

```
mvn -v
java -version
mvn clean
mvn compile
mvn test (if asked)
mvn package
mvn clean package (clean final build)
```

SUCCESS CHECK

A successful build normally ends with **BUILD SUCCESS**. For a web application, check **target/** for the generated WAR. For a standard Java application, check for the generated JAR.

3. MAVEN — TROUBLESHOOTING & FINAL CHECK

If Maven build fails

1	Read the error near the bottom of the Maven output first.
2	Open pom.xml and check dependencies, plugins and Java source/target settings.
3	Check whether Maven can resolve the required dependencies.
4	Run mvn dependency:tree if the problem involves dependencies or version conflicts.
5	Run mvn clean package -X when detailed build output is needed.
6	After making the required fix, repeat the build and confirm BUILD SUCCESS.

Local Maven repository (.m2)

Maven stores downloaded dependencies in the local repository. On Windows, the common location is:

```
%USERPROFILE%\.m2\repository
```

Build succeeds but application does not run

IMPORTANT

Separate **build problems** from **runtime/deployment problems**. If Maven says BUILD SUCCESS but the application fails, check the generated artifact, server/Tomcat configuration when applicable, port/context path, and application/server logs.

One-page memory sheet

```
1. CLONE → git clone <URL>
2. ENTER → cd <project>
3. CHECK → pom.xml
4. VERIFY → mvn -v / java -version
5. CLEAN → mvn clean
6. COMPILE → mvn compile
7. TEST → mvn test (if asked)
8. PACKAGE → mvn package
9. FINAL BUILD → mvn clean package
10. ARTIFACT → target/ (JAR or WAR)
11. RUN/DEPLOY → application server if required
12. VERIFY → browser/application output
```

REMEMBER THIS

Golden rule: Always know what the command is proving. **clean** removes old output → **compile** checks compilation → **test** checks tests → **package** creates the artifact → **target/** is where you verify the result.